RNN + LDA: New Frontiers in Sociological Research

Yufan Zhuang

Department of Applied Mathematics
Hong Kong Polytechnic University

April 17, 2018

Outline

1. Introduction
2. Recurrent Neural Networks
3. Latent Dirichlet Allocation
4. Implementation Details
5. Experimental Results
6. Summary

Outline

1. Introduction
2. Recurrent Neural Networks
3. Latent Dirichlet Allocation
4. Implementation Details
5. Experimental Results
6. Summary

Introduction

This capstone project aims to investigate unsupervised statistical methods used in sociological research, including

- Recurrent Neural Networks (RNN)
- Latent Dirichlet Allocation (LDA)

We will introduce both their theoretical foundations and real-world applications

Outline

1. Introduction
2. Recurrent Neural Networks
3. Latent Dirichlet Allocation
4. Implementation Details
5. Experimental Results
6. Summary

Recurrent Neural Networks

Basic Structure

Inspired by the structure of brain cells, that the current cell state depends on past states

$$h_t = \sigma_h(W_h v_t + U_h h_{t-1} + b_h)$$
$$y_t = \sigma_y(W_y h_t + b_y)$$

σ_h, σ_y: activation functions
$v_t \in \mathbb{R}^{D \times B}$
$W_h \in \mathbb{R}^{H \times D}$
$U_h, W_y \in \mathbb{R}^{H \times H}$
$h_t, y_t, b_h, b_y \in \mathbb{R}^{H \times B}$

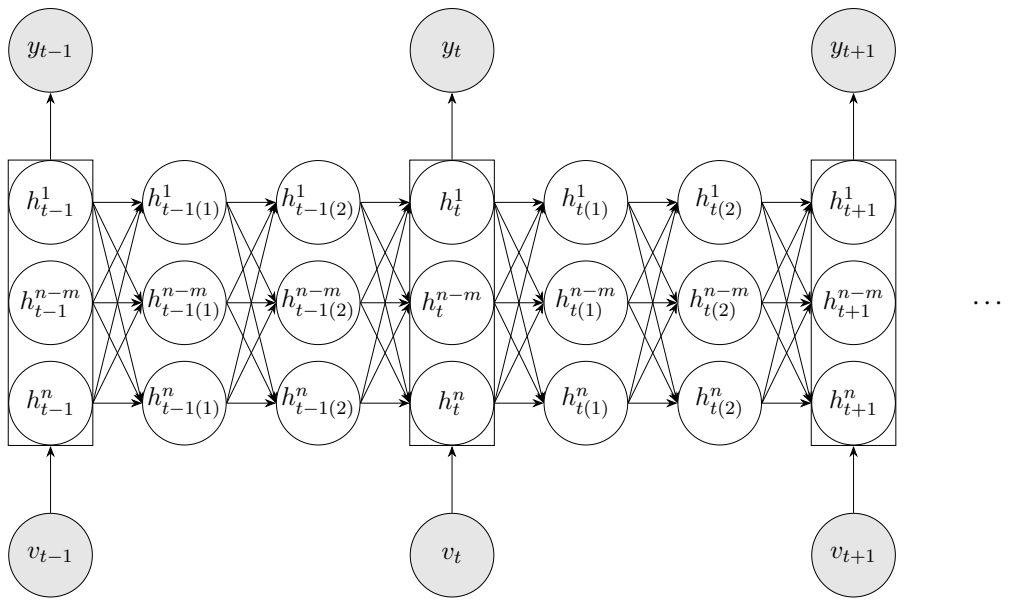


Figure: An Example of a RNN

H: dimension of hidden units, B: input batch size, D: input dimension

Recurrent Neural Networks

Difficulties in Optimization

- Very Deep Structure
  ∇_t would contain all terms from 1 to t

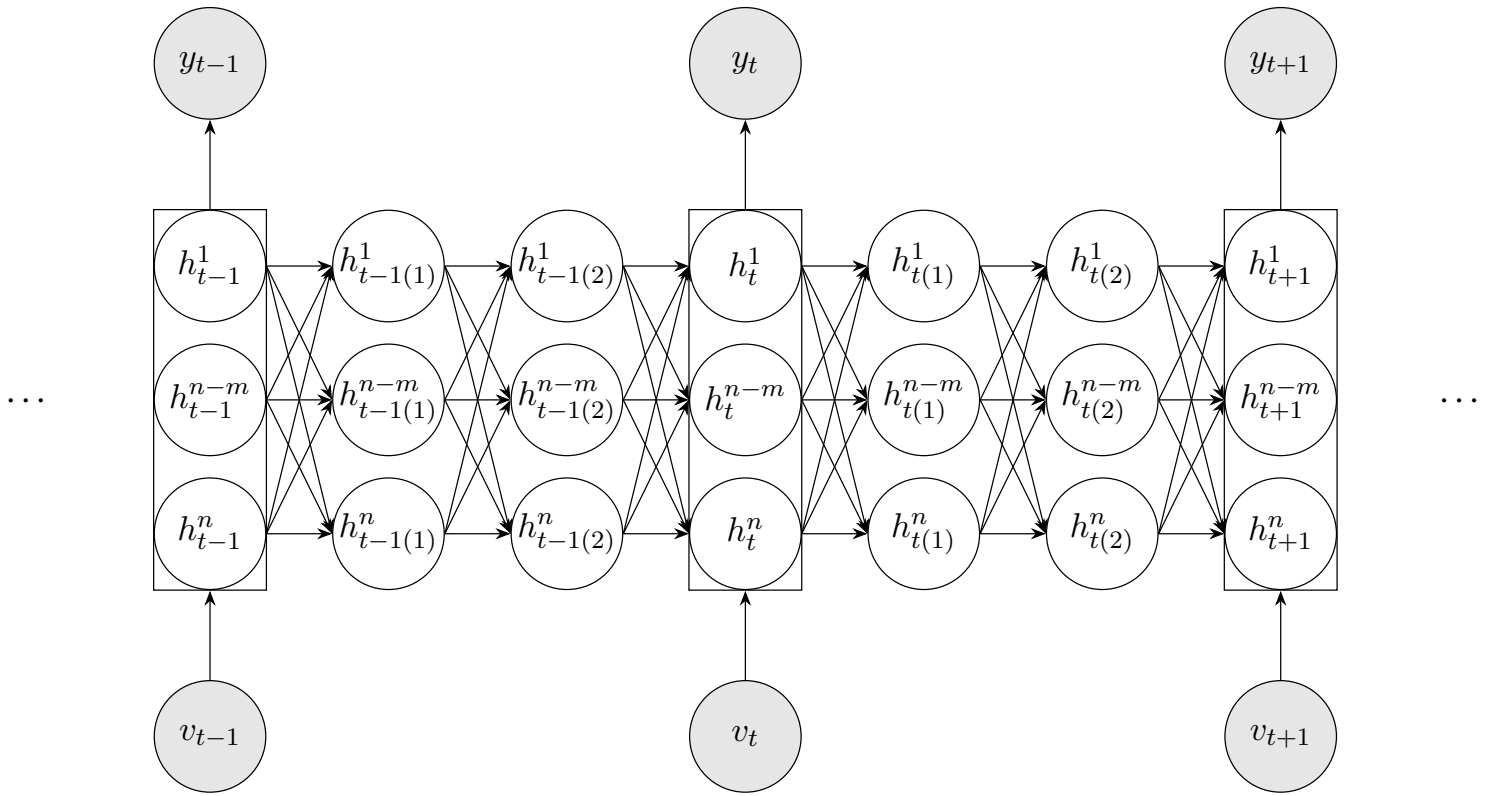


Figure: An Example of a RNN

- Vanishing Gradient Problem

$$\frac{\partial y_t}{\partial W_1} = \frac{\partial y_t}{\partial \sigma_t} \frac{\partial \sigma_t}{\partial \sigma_{t-1}} \cdots \frac{\partial \sigma_1}{\partial W_1}$$

where $\sigma'(x) \leq 1$ for most common activation functions (like $tanh(x)$).

$\frac{\partial y_t}{\partial W_1}$ tends to be very small when there are too many layers.

The network will stuck there.

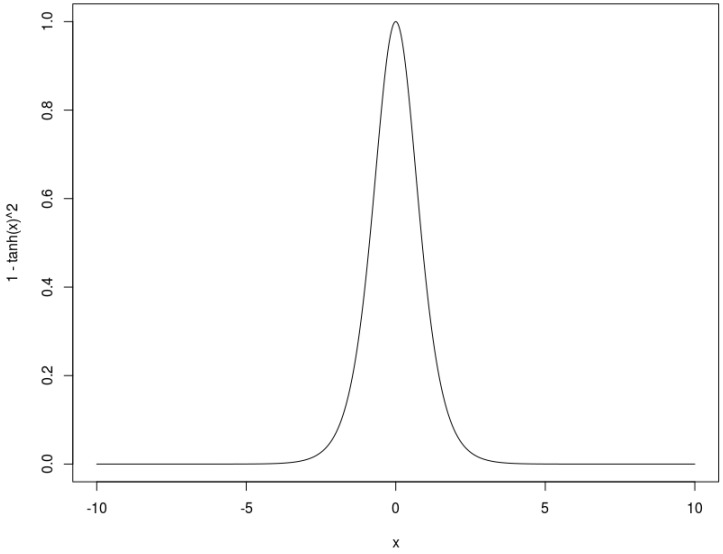


Figure: $tanh'(x)$

Recurrent Neural Networks

Tackling the Deep Structure

There are some popular optimization algorithms in optimizing RNN

- Stochastic Gradient Descent methods
  like adaptive moment estimation (also known as Adam)
  [Kingma and Ba, 2015]
- Unconventional Optimization Methods
  like optimizing neural networks with neural networks
  [Andrychowicz et al., 2016]

Recurrent Neural Networks

Tackling the Deep Structure

There are some popular optimization algorithms in optimizing RNN

- Stochastic Gradient Descent methods
  like adaptive moment estimation (also known as Adam)
  [Kingma and Ba, 2015]
- Unconventional Optimization Methods
  like optimizing neural networks with neural networks
  [Andrychowicz et al., 2016]

Recurrent Neural Networks

Stochastic Gradient Descent

Basic idea:

- Initialize the parameters and select proper hyper-parameters
- While not converge
  Shuffle examples and do gradient descent on mini-batches of data

Recurrent Neural Networks

Adam [Kingma and Ba, 2015]

Require: $\beta_1, \beta_2 \in [0, 1)$: exponential decay rates for the moment estimates.

Require: α : learning rate.

Require: $L(\theta)$: the objective function.

A suggestion about hyperparameters selection from the original paper is: $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$

while θ_t not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_\theta L_t(\theta_{t-1})$

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$

$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$

$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$

end while

Recurrent Neural Networks

Tackling the Vanishing Gradient Problem

Our human can choose what to memorize and what to forget, we hope RNN can do the same thing.

Two popular models:

- Long short-term memory (LSTM) [Hochreiter and Urgen Schmidhuber, 1997]
- Gated recurrent units (GRU) [Cho et al., 2014]

Recurrent Neural Networks

Long short-term memory (LSTM) [Hochreiter and Urgen Schmidhuber, 1997]

The structure of a LSTM neuron is shown in the figure, and the formulas are given below.

$$
\begin{aligned}
f_t &= \sigma(W_f \cdot x_t + b_f) \\
i_t &= \sigma(W_i \cdot x_t + b_i) \\
\tilde{c}_t &= tanh(W_c \cdot x_t + b_c) \\
c_t &= f_t * c_{t-1} + i_t * \tilde{c}_t \\
o_t &= \sigma(W_o \cdot x_t + b_o) \\
h_t &= o_t * tanh(c_t)
\end{aligned}
$$

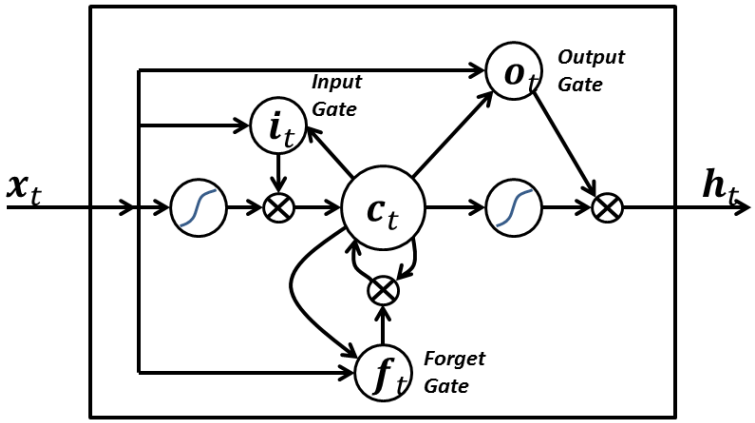


Figure: An Example of a LSTM Neuron (Source: Wikipedia)

Recurrent Neural Networks

Apply LSTM RNN on Peptide-binding Prediction

20 amino acids constitute all kinds of proteins that support most lives.

A specific Major Histocompatibility Complex (MHC) would bind some peptides but would not bind others.

Recurrent Neural Networks

Apply LSTM RNN on Peptide-binding Prediction

MHC: DRB1*0101

Benchmark: 84.6% correction rate (NetMHC IIpan-2.0 [Shen et al., 2014]).

The data is downloaded from the IEDB database[1] and the network is trained under the Tensorflow framework.

[1] http://www.cbs.dtu.dk/services/NetMHCcons/

Recurrent Neural Networks

Apply LSTM RNN on Peptide-binding Prediction

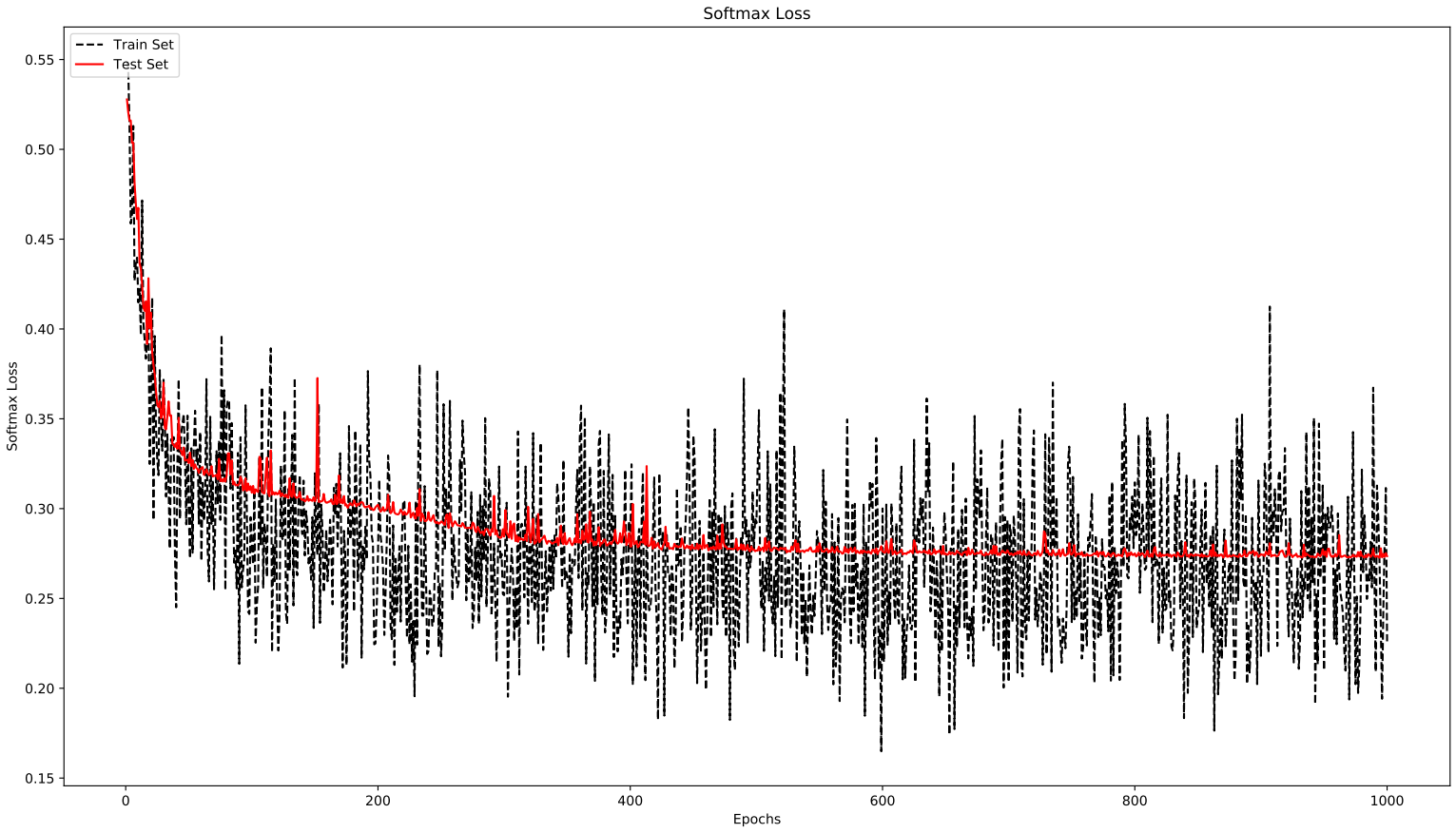


Figure: Softmax Loss versus Training Epoch

Recurrent Neural Networks

Apply LSTM RNN on Peptide-binding Prediction

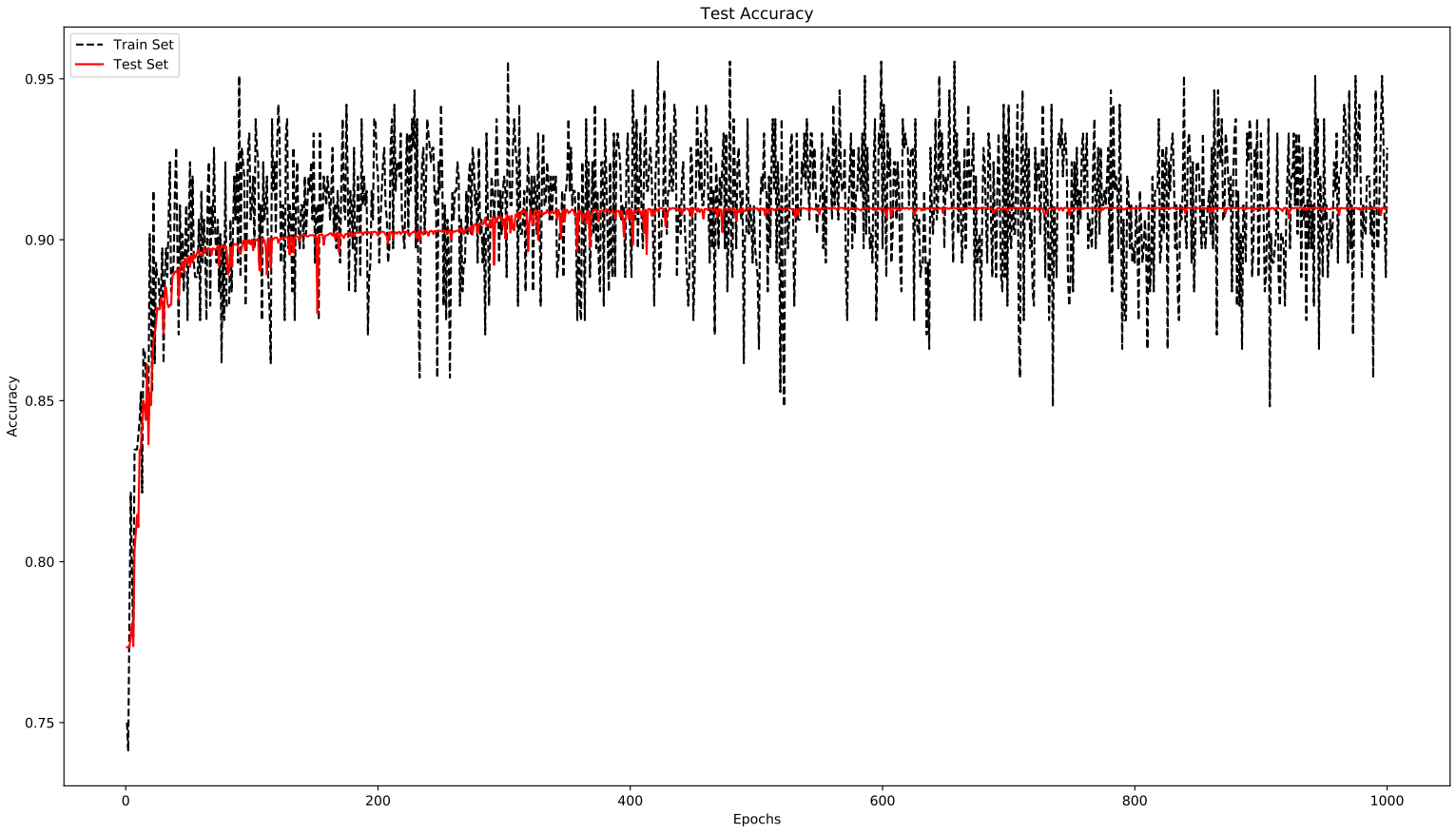


Figure: Accuracy versus Training Epoch

Recurrent Neural Networks

Sentiment Analysis

- Previous Approach: Bag-of-words model

	love	a	an	apple	banana	cat	happy
$text_1$	6	87837	787	28	72	84	99
$text_2$	7	78	5415	52	65	29	91
$text_3$	545	778	7507	90	76	25	12
$text_4$	545	18744	7560	32	35	61	77
$text_5$	88	788	6344	61	3	19	45

Table: An example of a bag-of-words representation

- Modern Approach: RNN
  multiplicative LSTM (mLSTM) [Krause et al., 2016]

Recurrent Neural Networks

mLSTM RNN [Krause et al., 2016]

The structure is given below, that one more intermediate state m_t is added in between neurons:

$$
\begin{aligned}
m_t &= (W_{mx} \cdot x_t) \odot (W_{mh} \cdot h_{t-1}) \\
\hat{h}_t &= W_{hx} \cdot x_t + W_{hm} \cdot m_t \\
i_t &= \sigma(W_{ix} x_t + W_{im} m_t) \\
o_t &= \sigma(W_{ox} x_t + W_{om} m_t) \\
f_t &= \sigma(W_{fx} x_t + W_{fm} m_t) \\
c_t &= f_t \odot c_{t-1} + i_t \odot \hat{h}_t \\
h_t &= tanh(c_t \odot o_t)
\end{aligned}
$$

$\odot$ denotes the Hadamard product (element-wise multiplication)

Recurrent Neural Networks

mLSTM RNN [Krause et al., 2016]

Often times, there is not enough labeled data for supervised learning, one solution for that is a two-layer model:

- Bottom Layer: Unsupervised RNN
  (Trained as a Predictive Network on the Byte-level)
- Top Layer: Logistic Regression

Performance on Stanford Sentiment Treebank

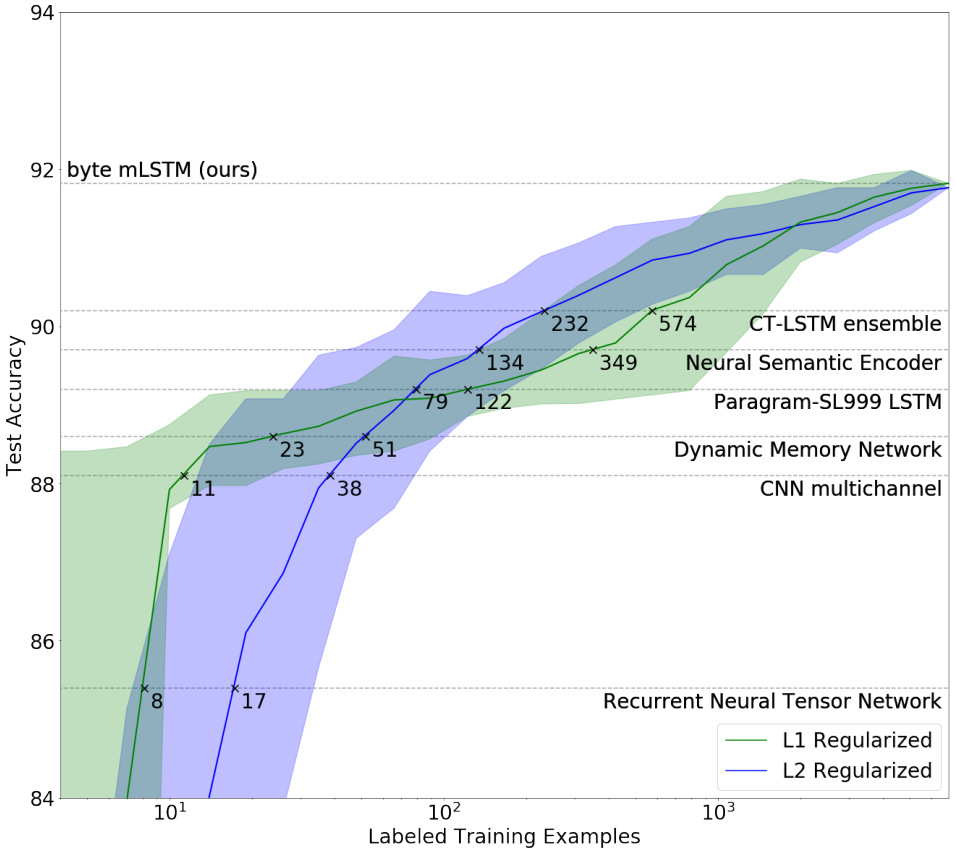


Figure: Performance Plots of the mLSTM Network (Source: [Radford et al., 2017])

Recurrent Neural Networks

mLSTM RNN [Krause et al., 2016]

Sentiment neuron: accuracy of 92.30% [Radford et al., 2017] on IMDB reviews.
Might due to L_1 regularization.

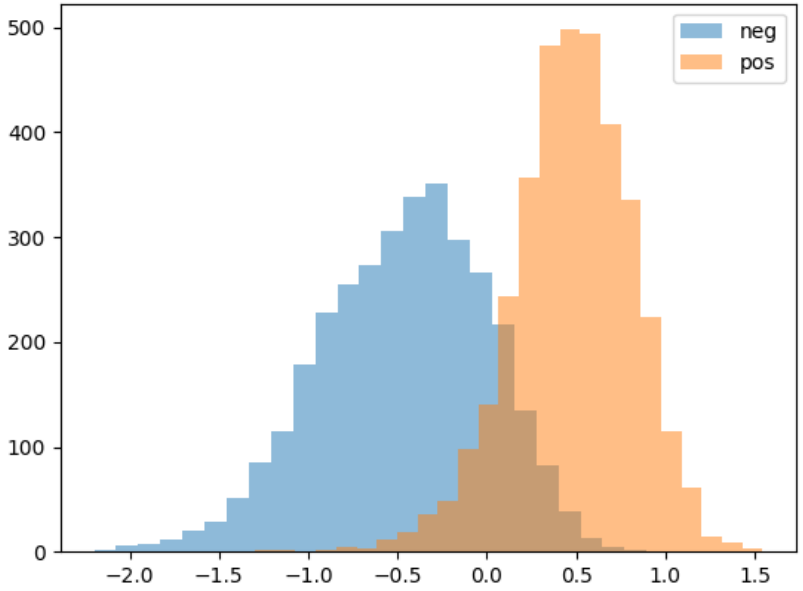


Figure: The Sentiment Unit (Source: [Radford et al., 2017])

Outline

1. Introduction
2. Recurrent Neural Networks
3. Latent Dirichlet Allocation
4. Implementation Details
5. Experimental Results
6. Summary

Latent Dirichlet Allocation

Probablistic Topic Models

Topic modeling is significantly boosted after the proposal of latent Dirichlet allocation [Blei et al., 2003, Hoffman et al., 2010].

Topic ID	Topic Content
0	death, sentence, usa, case, yesterday, execution, official, crime, embassy, penalty
1	human rights, prisoner, government, prison, dissident, political, usa, right, sentence, freedom
2	wei, beijing, li, hong kong, government, bethune, canada, usa, police, money
3	child, family, woman, parent, home, life, mother, son, father, girl

Topic modeling provides ways to organize documents automatically.

Latent Dirichlet Allocation

Theoretical Foundations

The Dirichlet random variable θ_d of k dimensions with hyper-parameter α takes values in the (k-1) simplex, and its density function is listed below:

$$p(\theta_d|\alpha) = \frac{\Gamma(\sum_{i=1}^{k} \alpha_i)}{\prod_{i=1}^{k} \Gamma(\alpha_i)} \theta_{d1}^{\alpha_1 - 1} \theta_{d2}^{\alpha_2 - 1} \dots \theta_{dk}^{\alpha_i - 1}$$

where $\forall i, \alpha_i > 0$, and $\Gamma(x)$ denotes the Gamma function ($\Gamma(x) = \int_0^\infty u^{x-1} e^{-u} du$).

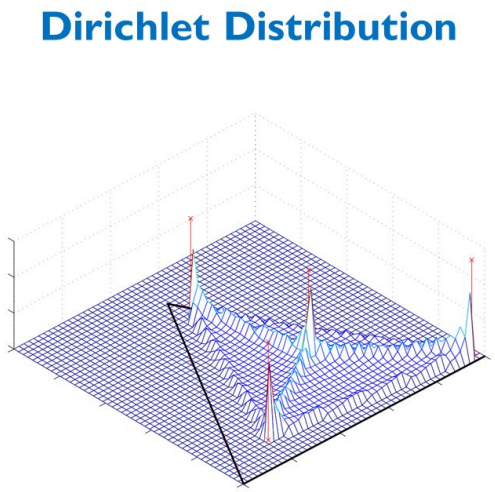


Figure: Dirichlet Distribution (Source: [Blei et al., 2003])

LDA in plate notation

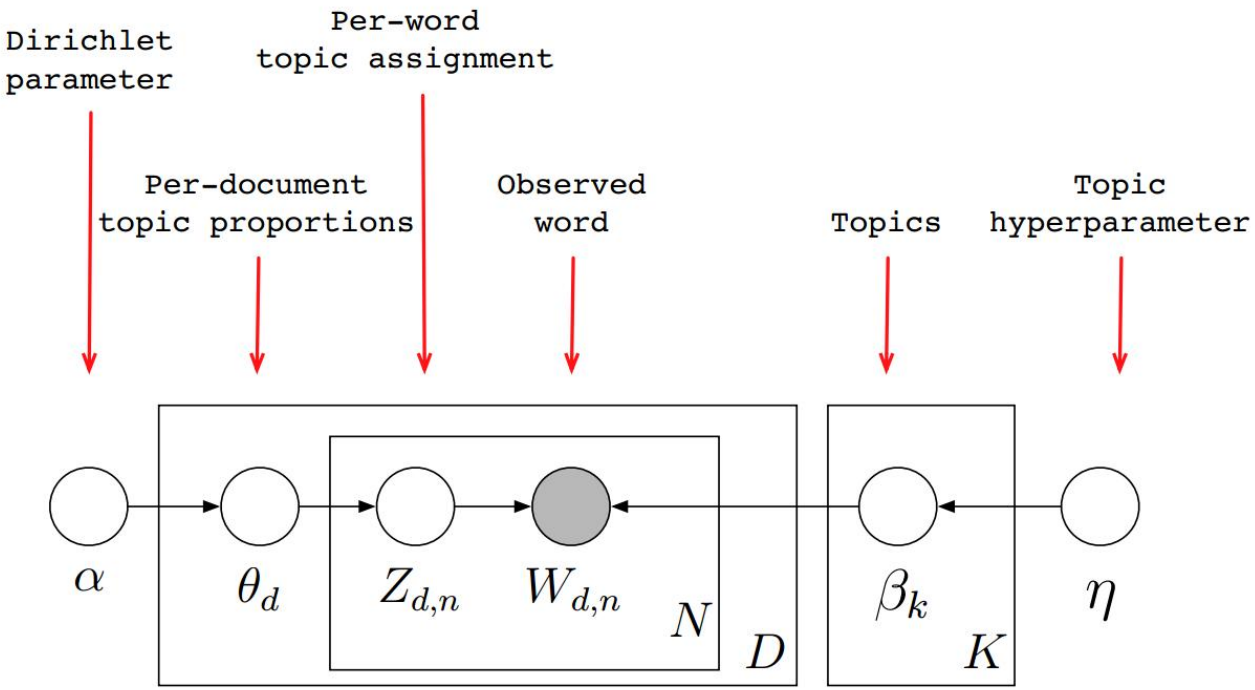


Figure: Latent Dirichlet Allocation (Source: Lecture notes of CPSC522, University of British Columbia)

Latent Dirichlet Allocation

Theoretical Foundations

Given hyper-parameters α and η, the joint distribution of the topic mixture θ, K distributions of topic β, topics for N words $\boldsymbol{z}$, and N words $\boldsymbol{w}$ is given by:

$$p(\theta, \beta, \boldsymbol{z}, \boldsymbol{w}|\alpha, \eta) = p(\theta|\alpha)p(\beta|\eta) \prod_{n=1}^{N} p(z_n|\theta)p(w_n|z_n, \beta)$$

Thus, the marginal distribution of a document can be written as:

$$p(\boldsymbol{w}|\alpha, \eta) = \int_{\beta} \int_{\theta} \sum_{z} p(\theta, \beta, \boldsymbol{z}, \boldsymbol{w}|\alpha, \eta)$$

by integrating over β, θ and $\boldsymbol{z}$.
Taking the product of all documents, the probability of a set of documents is:

$$p(D|\alpha, \eta) = \prod_{d=1}^{D} \int_{\beta} \int_{\theta} \sum_{z} p(\theta, \beta, \boldsymbol{z}, \boldsymbol{w}|\alpha, \eta)$$

Key inference problem: compute the posterior distribution when documents are given:

$$p(\theta, \beta, \boldsymbol{z}|\boldsymbol{w}, \alpha, \eta) = \frac{p(\theta, \beta, \boldsymbol{z}, \boldsymbol{w}|\alpha, \eta)}{p(\boldsymbol{w}|\alpha, \eta)}$$

It is not easy to solve because it will not be easy to integrate over θ along with β in its space.

$$p(\boldsymbol{w}|\alpha, \eta) = \int_\beta \int_\theta \sum_z p(\theta, \beta, \boldsymbol{z}, \boldsymbol{w}|\alpha, \eta)$$

Latent Dirichlet Allocation

Theoretical Foundations

Two major methods

- Variational Inference
- Markov chain Monte Carlo

Latent Dirichlet Allocation

Variational Inference

Key idea: select a family of distributions q over the latent variables with its own variational parameters [Blei et al., 2003].

$$q(\theta, \beta, \boldsymbol{z}; \gamma, \lambda, \phi) = q(\theta; \gamma) q(\beta; \lambda) q(\boldsymbol{z}; \phi)$$

where

$$q(\theta_d; \gamma_d) = Dir(\theta_d; \gamma_d); \quad q(\beta_k; \lambda_k) = Dir(\beta_k; \lambda_k); \quad q(z_{dn} = k) = \phi_{dnk}.$$

What we want: q is a close proxy for the posterior distribution p.
That is measured with Kullback-Leibler (KL) divergence.

The KL-divergence for the two distribution q and p is

$$KL(q||p) = \int q \cdot \log \frac{q(\theta, \beta, \boldsymbol{z})}{p(\theta, \beta, \boldsymbol{z}|\boldsymbol{w}, \alpha, \eta)} du$$

KL-divergence is always greater than zero because

$$
\begin{aligned}
KL(q||p) &= \int q \cdot \log \frac{q}{p} du \\
&= -\int q \cdot \log \frac{p}{q} du \\
&\geq -\log(\int q \cdot \frac{p}{q} du) \quad (Jensen's\ inequality) \\
&= -\log(\int p du) \\
&= 0
\end{aligned}
$$

But KL-divergence is not symmetric ($KL(q||p) \neq KL(p||q)$)

We aim to solve the following optimization problem

$$(\gamma^*, \lambda^*, \phi^*) = \arg min_{\gamma,\lambda,\phi} KL(q||p)$$

However, most of the time, the KL-divergence cannot be optimized directly.

Latent Dirichlet Allocation

Variational Inference

Instead, we minimize the evidence lower bound (ELBO) that is based on Jensen's inequality (for a convex function f, $f[E(x)] \leq E[f(x)]$).

$$
\begin{aligned}
\log(p(\boldsymbol{w}|\alpha,\eta)) &= \log \int_\beta \int_\theta \sum_z p(\theta,\beta,\boldsymbol{z},\boldsymbol{w}|\alpha,\eta) \\
&= \log \int_\beta \int_\theta \sum_z \frac{p(\theta,\beta,\boldsymbol{z},\boldsymbol{w}|\alpha,\eta)}{q(\theta,\beta,\boldsymbol{z})} q(\theta,\beta,\boldsymbol{z}) \\
&= \log(E_q[\frac{p(\theta,\beta,\boldsymbol{z},\boldsymbol{w}|\alpha,\eta)}{q(\theta,\beta,\boldsymbol{z})}]) \\
&\geq E_q[\log\ p(\theta,\beta,\boldsymbol{z},\boldsymbol{w}|\alpha,\eta)] - E_q[\log\ q(\theta,\beta,\boldsymbol{z})]
\end{aligned}
$$

and $\mathscr{L}(\boldsymbol{w},\gamma,\lambda,\phi) := E_q[\log\ p(\theta,\beta,\boldsymbol{z},\boldsymbol{w}|\alpha,\eta)] - E_q[\log\ q(\theta,\beta,\boldsymbol{z})]$ is defined as the ELBO.

Latent Dirichlet Allocation

Variational Inference

$$
\begin{aligned}
&KL(q||p) \\
&= \int q \cdot \log \frac{q(\theta, \beta, \boldsymbol{z})}{p(\theta, \beta, \boldsymbol{z}|\boldsymbol{w}, \alpha, \eta)} du \\
&= E_q[\log \frac{q(\theta, \beta, \boldsymbol{z})}{p(\theta, \beta, \boldsymbol{z}|\boldsymbol{w}, \alpha, \eta)}] \\
&= E_q[\log \frac{q(\theta, \beta, \boldsymbol{z}) p(\boldsymbol{w}|\alpha, \eta)}{p(\theta, \beta, \boldsymbol{z}, \boldsymbol{w}|\alpha, \eta)}] \\
&= E_q[\log\ p(\boldsymbol{w}|\alpha, \eta)] - (E_q[\log\ p(\theta, \beta, \boldsymbol{z}, \boldsymbol{w}|\alpha, \eta)] - E_q[\log\ q(\theta, \beta, \boldsymbol{z})]) \\
&= \log\ p(\boldsymbol{w}|\alpha, \eta) - \mathscr{L}(\boldsymbol{w}, \gamma, \lambda, \phi)
\end{aligned}
$$

Maximizing the ELBO = minimizing the KL-divergence.
Then we try to find the parameters that give the tightest bound on the $p(\theta, \beta, \boldsymbol{z}|\boldsymbol{w}, \alpha, \eta)$.

The primary goal of the original MCMC is to compute an intractable integral [Metropolis and Ulam, 1949].

$$J = \int \frac{F(\theta)\exp[-E(\theta)]/kT}{\exp[-E(\theta)]/kT} d\theta$$

where k denotes the Boltzmann constant, T denotes the system temperature, θ denotes N particles on $\mathbb{R}^2$, and the Boltzmann distribution $E(\theta)$ is defined as

$$E(\theta) = \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1(i\neq j)}^{N} V(d_{ij})$$

where d_{ij} is the Euclidean distance between two particles, V is an energy potential function.

MCMC says that we should sample more at places that have a higher probability density. At each step, it samples a particle in the system and proposes a random movement.

$$x_i = x_i + \sigma \cdot u_{1i}$$
$$y_i = y_i + \sigma \cdot u_{2i}$$

where u denotes uniform distributions in $[-1, 1]$, σ denotes the standard variance of particle movements.
Thus, a difference of energy $\triangle E$ is then computed. This proposal will be accepted with a probability of

$$min(1, \exp[-\triangle E]/kT)$$

Latent Dirichlet Allocation

Markov chain Monte Carlo

In [Griffiths and Steyvers, 2004], they derived the transition probability for a step in $\boldsymbol{z}$ as

$$P(z_i = j | z_{-i}, \boldsymbol{w}) \propto P(w_i \textit{ is from topic}_j) \cdot P(\textit{topic}_j \textit{ in current document})$$

but this can be generalized for all latent variables $\boldsymbol{z}$, β, and θ. The full conditionals are as follows

$$P(z_{dn} = k | \theta_d, \beta_k) \propto \exp(\log \theta_{dk} + \log \beta_{k,w_{dn}})$$

$$P(\theta_d | z_{dn} = k, \beta_k) = Dir(\alpha_k + \sum_{i=1}^{N} n(z_{di} = k))$$

$$P(\beta_k | z_{dn} = k, \theta_d) = Dir(\eta_n + \sum_{i=1}^{D} \sum_{j=1}^{N} n(w_{ij} = n, z_{ij} = k))$$

where n is a count that does not include the current assignment of variables.

Latent Dirichlet Allocation

A Simplified Version of Latent Dirichlet Allocation

In [Quinn et al., 2010], they proposed a simplified LDA with its prior modeled as a time series for modeling political texts. We modified the model as a more general case of the simplified LDA.

Document matrix $\boldsymbol{Y} \in \mathbb{R}^{D \times N}$
Latent variable matrix $\boldsymbol{Z} \in \mathbb{R}^{D \times K}$

Further assume that

$$Z_d \sim Multinomial(1, \pi_d),\ d = 1, ..., D.$$

$$Y_d \sim Multinomial(N_d, \theta_k)\ if\ Z_{dk} = 1,\ d = 1, ..., D,\ k = 1, ..., K.$$

The conjugate priors for $\boldsymbol{Z}$ and $\boldsymbol{Y}$ are

$$\pi_d \sim \textit{Dirichlet}(\alpha),\ \theta_k \sim \textit{Dirichlet}(\beta)$$

So the sampling density can be written as

$$p(\boldsymbol{Y}, \boldsymbol{Z}|\pi, \theta) \propto \prod_{d=1}^{D} \prod_{k=1}^{K} (\pi_{dk} \prod_{w=1}^{N} \theta_{kw}^{Y_{dw}})^{Z_{dk}}$$

We apply the EM algorithm to tackle this inference problem.

Let $\lambda := \{\pi, \theta\}, L(\lambda) := \log P(\boldsymbol{Y}|\lambda)$.
We aim to solve the following optimization problem

$$\lambda^* = \arg\max_{\lambda} L(\lambda)$$

For each iteration, we wish to find λ such that

$$L(\lambda) > L(\lambda^{(n)})$$

where $\lambda^{(n)}$ is the parameter obtained from the nth iteration.

Latent Dirichlet Allocation

A Simplified Version of Latent Dirichlet Allocation

Note that

$$P(\boldsymbol{Y}|\lambda) = \sum_{z} P(\boldsymbol{Y}|\boldsymbol{Z},\lambda)P(\boldsymbol{Z}|\lambda)$$

Thus

$$\begin{aligned}
L(\lambda) - L(\lambda^{(n)}) &= \log \sum_{z} [P(\boldsymbol{Y}|\boldsymbol{Z},\lambda)P(\boldsymbol{Z}|\lambda)] - \log P(\boldsymbol{Y}|\lambda^{(n)}) \\
&= \log \sum_{z} [P(\boldsymbol{Y}|\boldsymbol{Z},\lambda)P(\boldsymbol{Z}|\lambda)\frac{P(\boldsymbol{Z}|\boldsymbol{Y},\lambda^{(n)})}{P(\boldsymbol{Z}|\boldsymbol{Y},\lambda^{(n)})}] - \log P(\boldsymbol{Y}|\lambda^{(n)}) \\
&= \log \sum_{z} [P(\boldsymbol{Z}|\boldsymbol{Y},\lambda^{(n)})\frac{P(\boldsymbol{Y}|\boldsymbol{Z},\lambda)P(\boldsymbol{Z}|\lambda)}{P(\boldsymbol{Z}|\boldsymbol{Y},\lambda^{(n)})}] - \log P(\boldsymbol{Y}|\lambda^{(n)}) \\
&\geq \sum_{z} [P(\boldsymbol{Z}|\boldsymbol{Y},\lambda^{(n)}) \log \frac{P(\boldsymbol{Y}|\boldsymbol{Z},\lambda)P(\boldsymbol{Z}|\lambda)}{P(\boldsymbol{Z}|\boldsymbol{Y},\lambda^{(n)})}] - \log P(\boldsymbol{Y}|\lambda^{(n)}) \\
&= \sum_{z} [P(\boldsymbol{Z}|\boldsymbol{Y},\lambda^{(n)}) \log \frac{P(\boldsymbol{Y}|\boldsymbol{Z},\lambda)P(\boldsymbol{Z}|\lambda)}{P(\boldsymbol{Z}|\boldsymbol{Y},\lambda^{(n)})P(\boldsymbol{Y}|\lambda^{(n)})}]
\end{aligned}$$

We hope to maximize $\Delta(\lambda|\lambda^{(n)})$ in each iteration, that is

$$\begin{aligned}
\lambda^{(n+1)} &= \arg\max_{\lambda}\{\Delta(\lambda|\lambda^{(n)})\} \\
&= \arg\max_{\lambda}\{\sum_{z} P(\boldsymbol{Z}|\boldsymbol{Y},\lambda^{(n)}) \log \frac{P(\boldsymbol{Y}|\boldsymbol{Z},\lambda)P(\boldsymbol{Z}|\lambda)}{P(\boldsymbol{Z}|\boldsymbol{Y},\lambda^{(n)})P(\boldsymbol{Y}|\lambda^{(n)})}\} \\
&= \arg\max_{\lambda}\{\sum_{z} P(\boldsymbol{Z}|\boldsymbol{Y},\lambda^{(n)}) \log P(\boldsymbol{Y}|\boldsymbol{Z},\lambda)P(\boldsymbol{Z}|\lambda)\} \\
&= \arg\max_{\lambda}\{\sum_{z} P(\boldsymbol{Z}|\boldsymbol{Y},\lambda^{(n)}) \log \frac{P(\boldsymbol{Y},\boldsymbol{Z},\lambda)}{P(\boldsymbol{Z},\lambda)}\frac{P(\boldsymbol{Z},\lambda)}{P(\lambda)}\} \\
&= \arg\max_{\lambda}\{\sum_{z} P(\boldsymbol{Z}|\boldsymbol{Y},\lambda^{(n)}) \log P(\boldsymbol{Y},\boldsymbol{Z}|\lambda)\} \\
&= \arg\max_{\lambda}\{E_{\boldsymbol{Z}|\boldsymbol{Y},\lambda^{(n)}}[\log P(\boldsymbol{Y},\boldsymbol{Z}|\lambda)]\}
\end{aligned}$$

Latent Dirichlet Allocation

A Simplified Version of Latent Dirichlet Allocation

Denote the marginal probability of $P(Z_{dk} = 1 | \boldsymbol{Y}, \pi^{(t)}, \theta^{(t)})$ as $h_{dk}^{(t)}$

$$h_{dk}^{(t)} = \frac{\pi_{dk}^{(t)} (\prod_{w=1}^{N} \theta_{kw}^{(t)\,Y_{dw}})}{\sum_{k=1}^{K} [\pi_{dk}^{(t)} (\prod_{w=1}^{N} \theta_{kw}^{(t)\,Y_{dw}})]}$$

Thus

$$Q(\pi, \theta | \pi^{(t)}, \theta^{(t)}) = \sum_{d=1}^{D} \sum_{k=1}^{K} h_{dk}^{(t)} [log(\pi_{dk}) + \sum_{w=1}^{N} Y_{dw} log(\theta_{kw})]$$

In the M-step of the EM algorithm

$$\pi, \theta = \arg\max_{\pi, \theta} \; Q(\pi, \theta | \pi^{(t)}, \theta^{(t)})$$

We form the Lagrangian to solve this problem

$$\mathcal{L} = Q(\pi, \theta | \pi^{(t)}, \theta^{(t)}) - \sum_{d=1}^{D} \lambda_d (\sum_{k=1}^{K} \pi_{dk} - 1) - \sum_{k=1}^{K} \mu_k (\sum_{w=1}^{N} \theta_k - 1)$$

According to the KKT conditions, we have the following system of equations

$$\frac{\partial \mathcal{L}}{\partial \pi_{dk}} = \frac{h_{dk}^{(t)}}{\pi_{dk}} - \lambda_d = 0 \tag{1}$$

$$\frac{\partial \mathcal{L}}{\partial \theta_{kw}} = \frac{\sum_{d=1}^{D} h_{dk}^{(t)} Y_{dw}}{\theta_{kw}} - \lambda_d = 0 \tag{2}$$

$$\frac{\partial \mathcal{L}}{\partial \lambda_d} = \sum_{k=1}^{K} \pi_{dk} - 1 = 0 \tag{3}$$

$$\frac{\partial \mathcal{L}}{\partial \mu_k} = \sum_{w=1}^{N} \theta_{kw} - 1 = 0 \tag{4}$$

After solving this system of equations, we have

$$\pi_{dk} = \frac{h_{dk}^{(t)}}{\sum_{k=1}^{K} h_{dk}^{(t)}}, \quad \theta_{kw} = \frac{\sum_{d=1}^{D} h_{dk}^{(t)} Y_{dw}}{\sum_{w=1}^{N} \sum_{d=1}^{D} h_{dk}^{(t)} Y_{dw}}$$

for the MLE update scheme, same procedure for the MAP.

Outline

1. Introduction
2. Recurrent Neural Networks
3. Latent Dirichlet Allocation
4. Implementation Details
5. Experimental Results
6. Summary

Implementation Details

Data Acquisition and Preprocessing

Dataset

- Content: 50,000 China-related articles
- Source: the Washington Post (US), the New York Times (US), the Toronto Star (CAN) and the Globe and Mail (CAN)
- Time span: 1986-2016

Preprocess

- removing symbols and figures
- removing stop-words (words like “a",“an",“the",“of") with the natural language toolkit (NLTK) [Loper and Bird, 2004]
- Filtering both high-frequency words and low-frequency words
- Phrase Detection with rapid automatic keyword extraction (RAKE) [Rose et al., 2010]

Implementation Details

Data Acquisition and Preprocessing

Dataset

- Content: 50,000 China-related articles
- Source: the Washington Post (US), the New York Times (US), the Toronto Star (CAN) and the Globe and Mail (CAN)
- Time span: 1986-2016

Preprocess

- removing symbols and figures
- removing stop-words (words like “a",“an",“the",“of") with the natural language toolkit (NLTK) [Loper and Bird, 2004]
- Filtering both high-frequency words and low-frequency words
- Phrase Detection with rapid automatic keyword extraction (RAKE) [Rose et al., 2010]

Implementation Details

Model Evaluation

Key issue: need to determine the number of topics K.

Two approaches

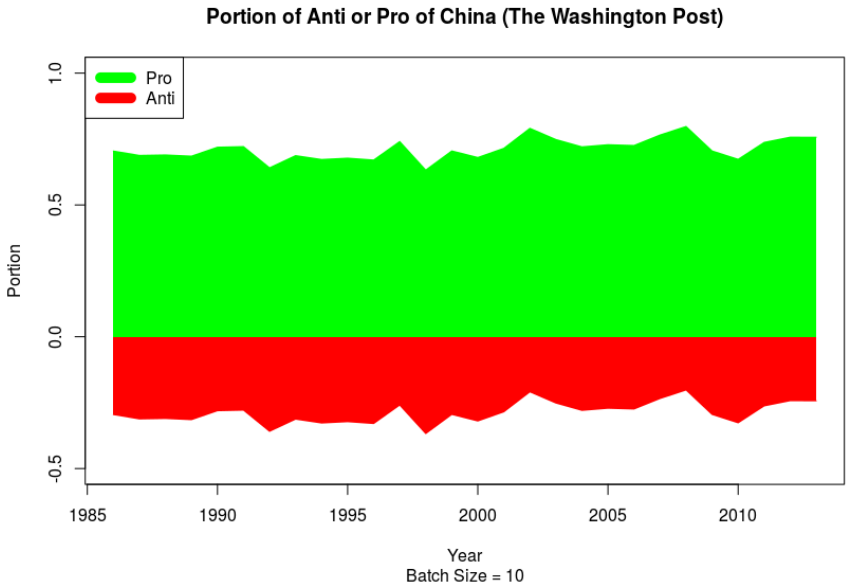
- likelihood based measure (perplexity)
- semantic coherence (C_{uci}, U_{mass}, C_{npmi} and C_v)
  words in same topic should appear together

Outline

1. Introduction
2. Recurrent Neural Networks
3. Latent Dirichlet Allocation
4. Implementation Details
5. Experimental Results
6. Summary

Experimental Results

Sentiment Analysis Output



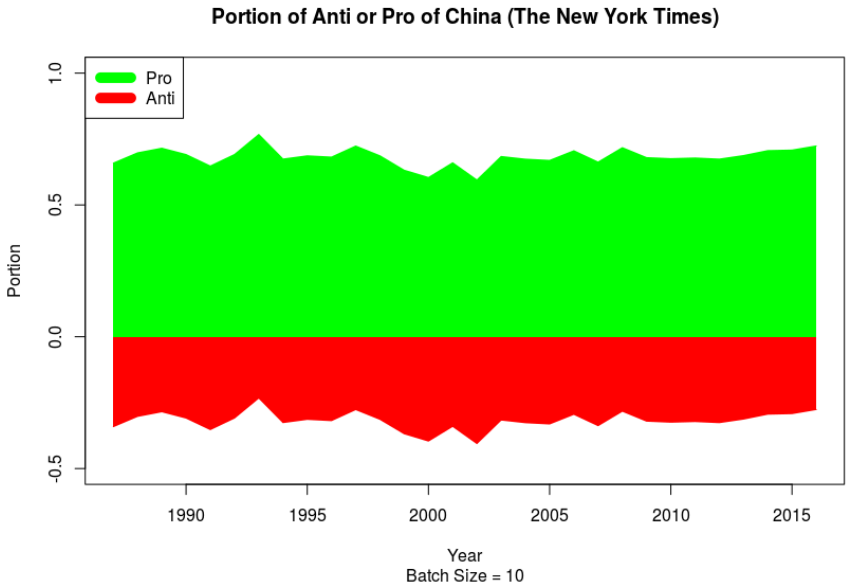
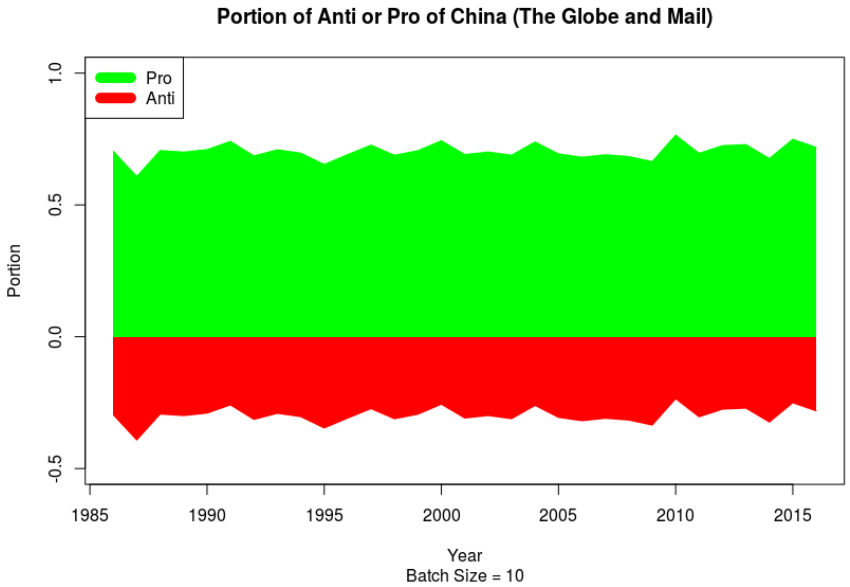


Figure: The outputs of the two major US newspapers

Experimental Results

Sentiment Analysis Output



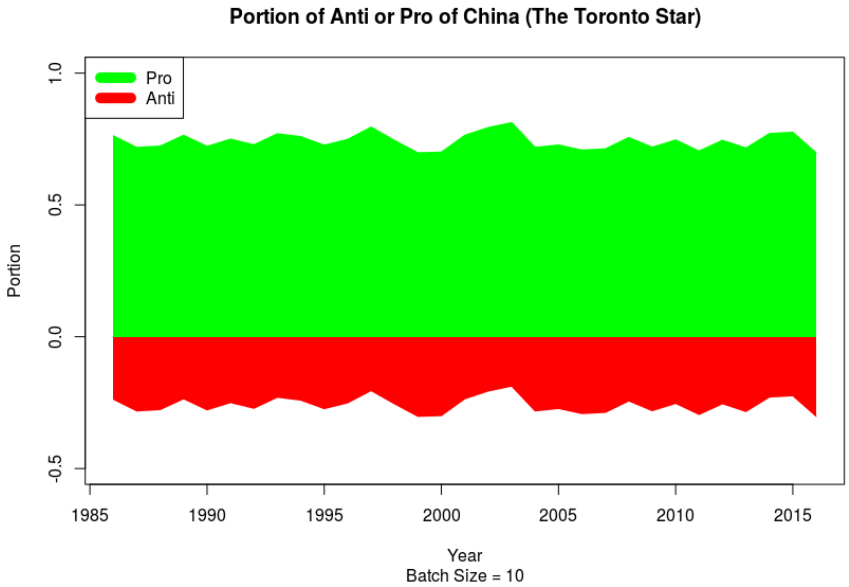
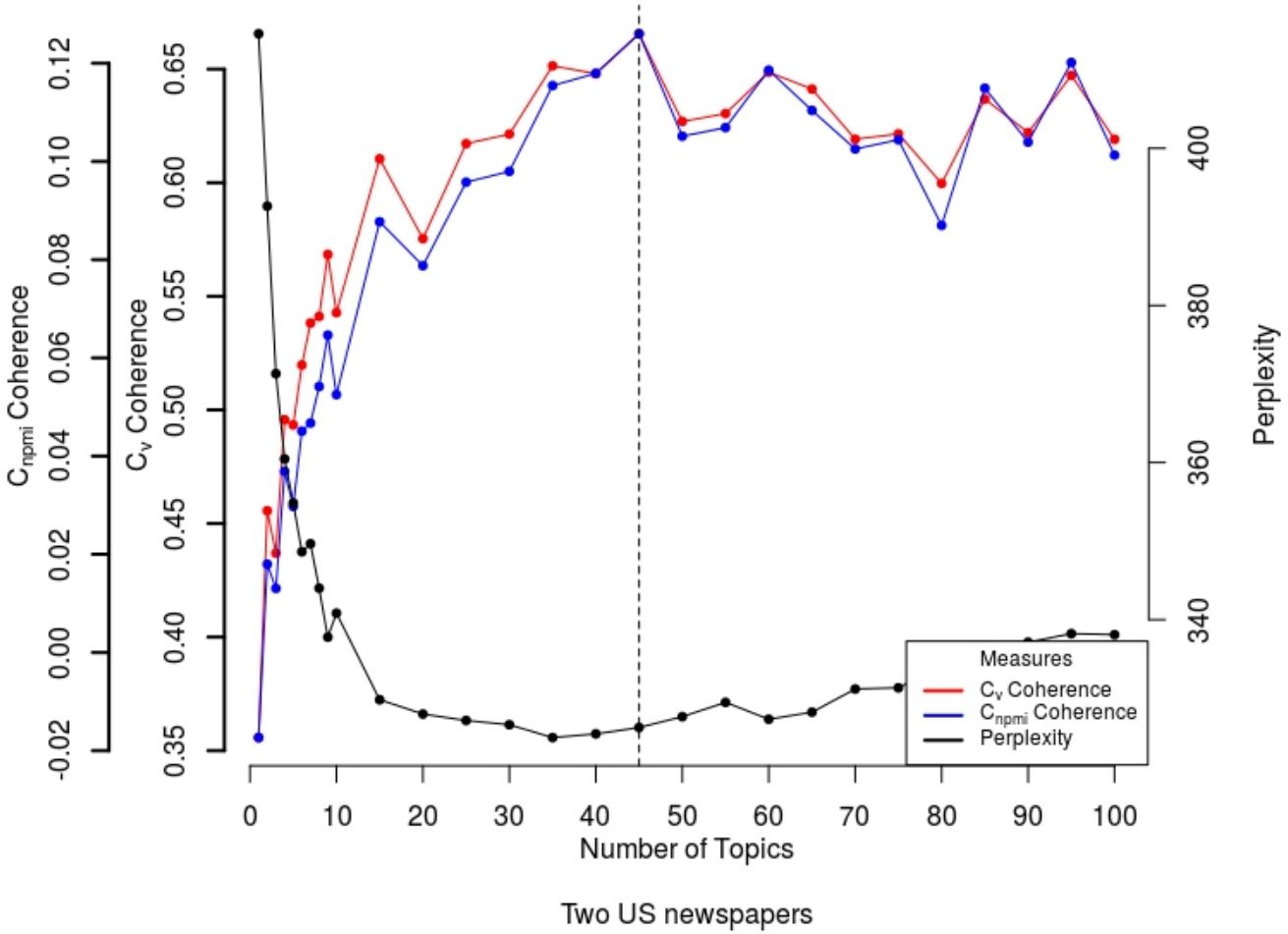


Figure: The outputs of the two major Canadian newspapers

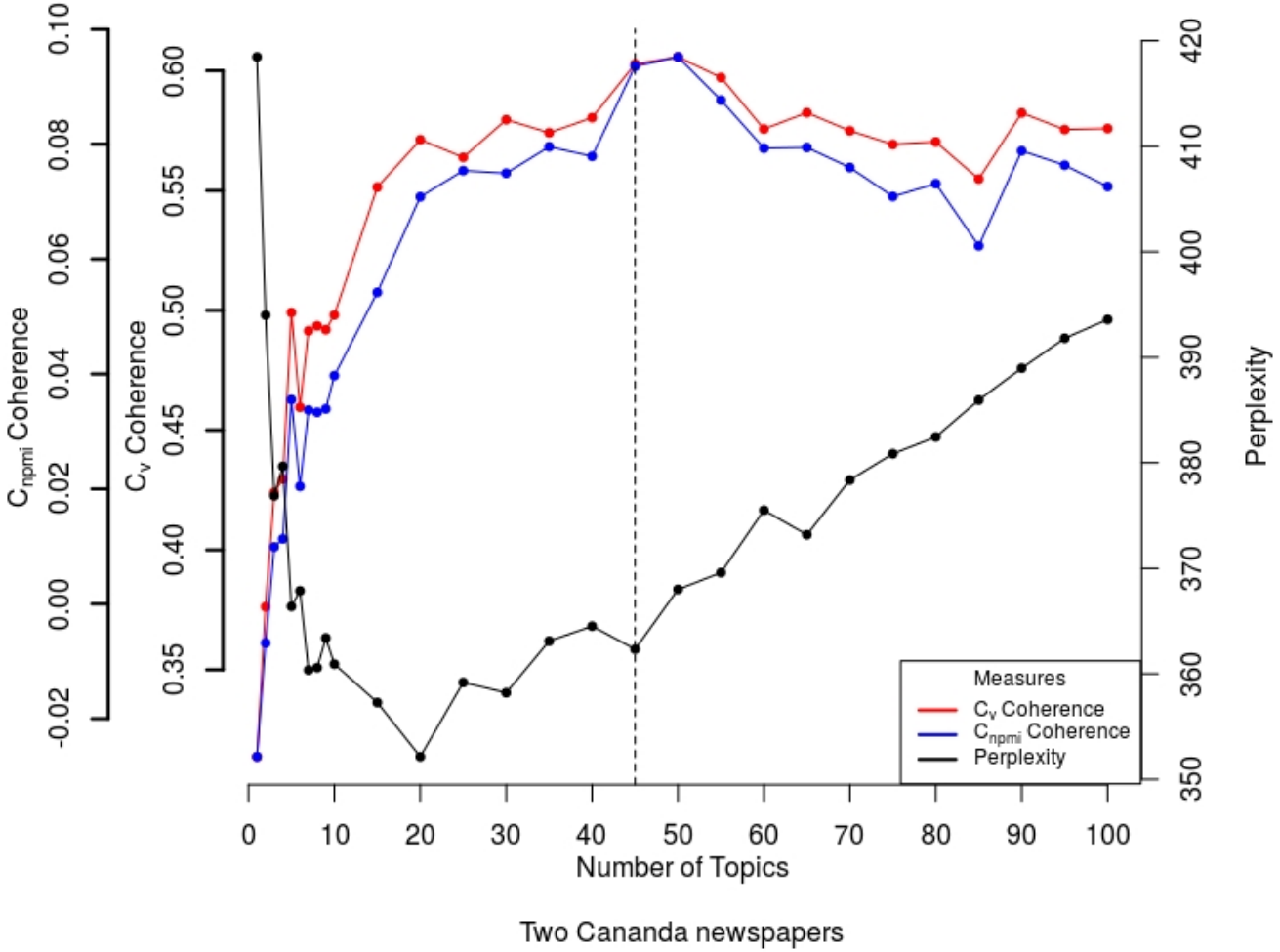
Experimental Results

LDA Model Evaluation and Selection



Experimental Results

LDA Model Evaluation and Selection



Outline

1. Introduction
2. Recurrent Neural Networks
3. Latent Dirichlet Allocation
4. Implementation Details
5. Experimental Results
6. Summary

Summary

- The RNN's theory and its appications in peptide-binding prediction and sentiment analysis.
- The LDA and ways to optimize it including variantional inference, MCMC and EM.
- Outlook
  - Latent trajectory analysis or network analysis to combine these two methods together.
  - Determine the optimal number of topics, K, with cluster analysis or eigenanalysis, in a more systematic and faster way.

Andrychowicz, M., Denil, M., Gómez, S., Hoffman, M. W., Pfau, D., Schaul, T., and de Freitas, N. (2016).
Learning to learn by gradient descent by gradient descent.
In *Proceeding of the 30th Conference on Neural Information Processing Systems (NIPS 2016)*, pages 3981–3989.

Blei, D. M., Edu, B. B., Ng, A. Y., Edu, A. S., Jordan, M. I., and Edu, J. B. (2003).
Latent Dirichlet Allocation.
Journal of Machine Learning Research, 3:993–1022.

Cho, K., van Merrienboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014).
Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation.
In *Proceeding of the Empirical Methods in Natural Language Processing (EMNLP 2014)*.

Griffiths, T. L. and Steyvers, M. (2004).
Finding scientific topics.

Proceedings of the National Academy of Sciences, 101(Supplement 1):5228–5235.

Hochreiter, S. and Urgen Schmidhuber, J. (1997).
LONG SHORT-TERM MEMORY.
Neural Computation, 9(8):1735–1780.

Hoffman, M. D., Blei, D. M., and Bach, F. (2010).
Online Learning for Latent Dirichlet Allocation.
Advances in Neural Information Processing Systems, 23:1–9.

Kingma, D. P. and Ba, J. (2015).
Adam: A Method for Stochastic Optimization.
In *Proceedings of the International Conference on Learning Representations 2015*.

Krause, B., Murray, I., Renals, S., and Lu, L. (2016).
MULTIPLICATIVE LSTM FOR SEQUENCE MODELLING.
In *ICLR 2017 Workshop track*.

Loper, E. and Bird, S. (2004).

NLTK: The Natural Language Toolkit.
In *Proceedings of the ACL 2004 on Interactive poster and demonstration sessions*, page 31.

Metropolis, N. and Ulam, S. (1949).
The Monte Carlo Method.
Journal of American Statistical Association, 44(247):335–341.

Quinn, K. M., Monroe, B. L., Colaresi, M., Crespin, M. H., and Radev, D. R. (2010).
How to analyze political attention with minimal assumptions and costs.
American Journal of Political Science, 54(1):209–228.

Radford, A., Jozefowicz, R., and Sutskever, I. (2017).
Learning to Generate Reviews and Discovering Sentiment.

Rose, S., Engel, D., Cramer, N., and Cowley, W. (2010).
Automatic Keyword Extraction from Individual Documents.
In *Text Mining: Applications and Theory*, pages 1–20. John Wiley & Sons, Ltd, Chichester, UK.

Shen, W. J., Wei, Y. T., Guo, X., Smale, S., Wong, H. S., and Li, S. C. (2014).
MHC binding prediction with KernelRLSpan and its variations.
Journal of Immunological Methods, 406:10–20.